Blind (Successfully?) Leading the Blind: Collaborative LLM Reasoning in Lean Theorem Proving

Henry Wang, Tom Liu
henwang@stanford.edu, liumaizi@stanford.edu
Stanford Computer Science

Project Overview

Background:

- Lean requires proofs that are both mathematically correct and accepted by a formal logical verifier
- LLMs can reason about math but often fail to generate Lean proofs due to invalid tactics, syntax errors, and hallucinated theorem. i.e. **LLMs aren't great at Lean**

Motivation:

- Chain of Thought reasoning improves performance by **generating intermediate reasoning steps** [1]
- Tree of Thoughts expands on it by **exploring multiple reasoning paths** and selecting best candidates [2]
- Iterative LLM feedback can improve performance **even when the evaluator cannot independently solve the problem**

Research Question:

- Can **search-based prompting** and **LLM self-evaluation** can improve performance in Lean theorem proving?

Datasets & Metrics

- **Dataset:** LeanDojo v2 Mathlib dataset containing 126 theorems specified in Lean (example input)

```
"∀ (n : Int) (d a : Nat) (a0 : a ≠ 0),
  mkRat (n * ↑a) (d * a) = mkRat n d"
```

In English: $\frac{na}{da} = \frac{n}{d}$ when $a \neq 0$

- **LLM Output:** List of Lean tactics to prove theorem

$$\text{Accuracy} = \frac{\text{Number of solved theorems}}{\text{Total number of theorem problems}}$$

- **Solved** only if **verified** in Lean via Pantograph

Example Output

```
intro n d a a0
by_cases hd : d = 0
· rw [hd]
simp [hd, Nat.zero_mul]
· rw [Rat.mkRat_eq_iff (Nat.mul_ne_zero hd a0) hd]
ring
ac_rfl
```

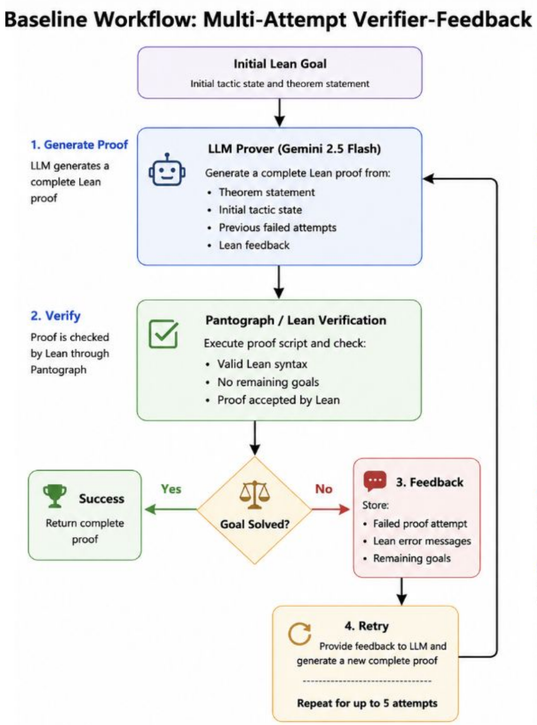
Interpretation:

1. Split into cases d = 0 and d ≠ 0
2. Trivial simplification in the d = 0 case
3. Use cross-multiplication to show equality
4. Simplify the resulting algebraic identity

Methods

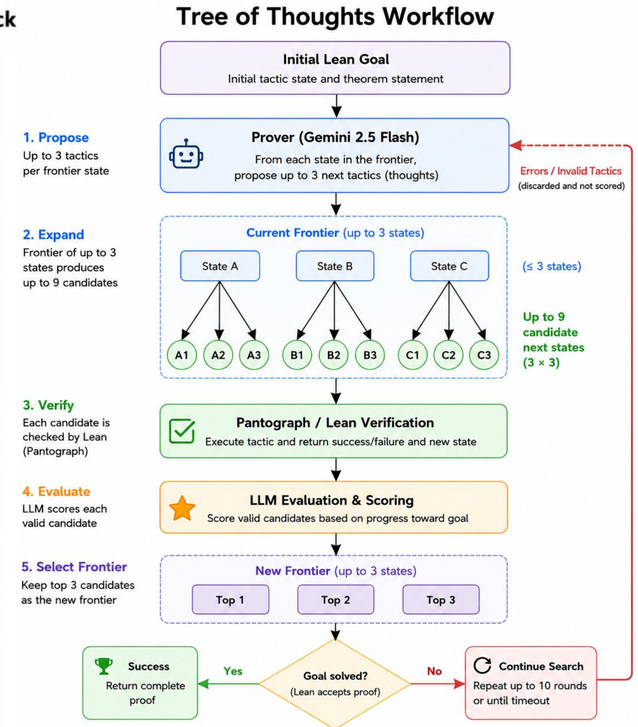
Baseline: Multi-Attempt Verifier Feedback

1. LLM generates a complete Lean proof
2. Lean/Pantograph verifies the proof
3. If verification fails, Lean errors and previous attempts are fed back to the LLM
4. Repeat for up to 5 attempts



Tree of Thoughts

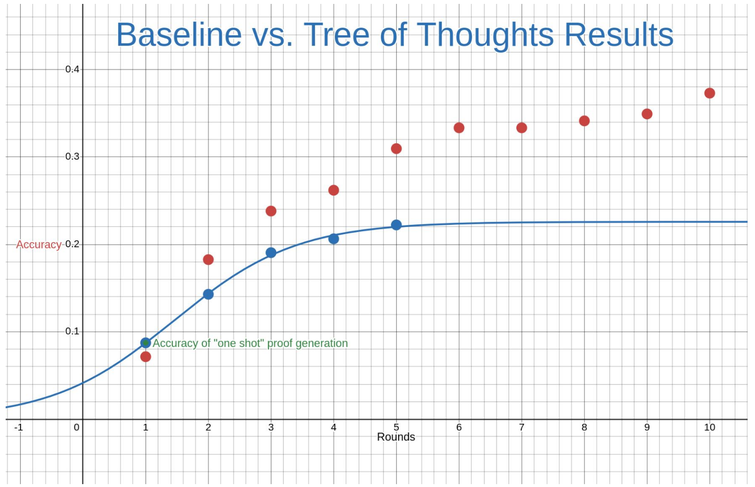
1. Prover generates 3 candidate tactics (ideas move the proof forward)
2. Lean verifies tactics and updates proof states.
3. Evaluator scores candidates using Lean feedback and current proof state
4. Top 3 candidates are retained



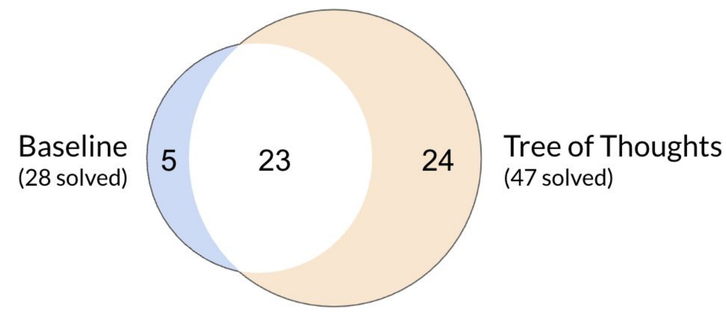
Results

Results:

- Incorporating Lean and LLM-based evaluation feedback loop (5 rounds) improved performance by 54%, from 11 to **28 (22.2%)**
- Utilizing Tree-of-Thoughts further improved to **47 (37.3%)** solved theorems
- Even with five attempts, Tree of Thoughts solved 39 (31.0%) problems solved, **39.3%** increase from baseline with 5 attempts



Overlap of Solved Theorems



- Tree of Thoughts largely **subsumed** most of problems solved by the baseline, suggesting that structured search retained the strengths of verifier feedback while overcoming some of its limitations.

Discussions & Future Research

Discussions:

- The baseline approach shows that as we add more tokens/attempts, performance **increases but seems to plateau** at 23% (logistic regression (r^2 = 0.9974) on baseline data plateaus at 23%).
- Our Tree of Thoughts approach achieves 37.3%, beating the baseline plateau by 14%.
- We hypothesize that Tree of Thoughts is able to outperform the baseline approach because of **task decomposition**, namely decomposing each theorem into tactics and exploring multiple reasoning paths

Future Research:

- Can generalize Tree of Thoughts with Graph of Thoughts, which allows separate ideas to merge
- Lean v4.29.1 was released **nearly a year** after Gemini 2.5 Flash (4/14/26 vs. 6/17/25). Finetuning the LLM could reduce hallucinated Lean tactics **(which was biggest problem in our model)**

References

[1] Wei et al., 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models.
[2] Yao et al., 2023. Tree of Thoughts: Deliberate Problem Solving with Large Language Models.
[3] Madaan et al., 2023. Self-Refine: Iterative Refinement with Self-Feedback.